第21章 不相交集合：練習

21-E01 Partition 與可重複 UNION

六點a..f依序加入ab、cd、bc、ab、ef、de；列每步partition、成功合併次數及集合數。若MAKE-SET全做完且每條邊呼叫UNION一次，每個UNION內做兩次FIND，總外部API數m為何？若用相同根仍執行LINK且rank相同加一，會破壞哪個一般不變量？

21-E02 Naive 串列的真正成本

先建立$x_1..x_6$，再依UNION(x_2,x_1)、UNION(x_3,x_2)、…、UNION(x_6,x_5)操作。對「總把第二串列接到第一尾端」及「短接長、同長保留第一」各列改set人數、最後代表及總改寫。一般n時證前者平方、後者此特定序列線性；說明後者並非所有序列都線性。

21-E03 八元素 weighted list 的逐人成長

同長保留第一，依(a,b)、(c,d)、(e,f)、(g,h)、(a,c)、(e,g)、(a,e)合併。列a..h各自改set次數、h每次改寫後的集合大小及總數。給$n=2^k$的平衡合併一般總改寫式，並證每個成員改寫至多floor($\log_2 n$)。

21-E04 同一組操作的 forest

改用rank、同rank保留第一根，執行21-E03次序。列最終全部parent與rank，追蹤FIND(h)的兩趟與後續FIND(d)。若二趟版先做parent[x]=root，再用x=parent[x]繼續，漏改哪些點？是否可以在find中把rank清成目前depth？

21-E05 Rank 不是目前高度下界

構造rank=2的根目前高度只有1的合法例子，寫出所需UNION與FIND。證明任一根的集合大小至少2^r及任一rank≤floor($\log_2 n$)，並解釋非根的目前subtree即使縮小仍不減rank的原因。$n=10^6$時rank至多多少，儲存rank值本身至少需多少bits？

21-E06 level 與 iter 的實數值

取n=16、α=3，balanced rank樹中find path依h(rank0)→g(rank1)→e(rank2)→p(rank3)→r(rank4)，根r自指。計算g,e,p壓縮前後的level、iter、φ，以及這條path的Φ變化；哪些是好節點、哪些是例外？推導$A_1(j)$、$A_2(j)$、$A_3(1)$，說明A_k的上標為何不是數值次方。

21-E07 位能不能只寫大 O 相消

假設一次find有s=7個節點、α=3，其中至少g=2個好節點；實際成本≤2s，採Ψ=3Φ。求該次攤銷上界。一般s時證以固定C>c可得O(1+α)。另有四次操作實際成本(3,7,2,8)，Ψ從0依序為(4,2,9,6)，列攤銷成本與總帳，驗證望遠鏡方向。

第21章 不相交集合：練習 （續）

21-E08 單次、總成本及小 n

分別比較(a)rank但不壓縮、(b)任意LINK且未曾find的鏈、(c)rank加壓縮的單次find與總成本。給可達$\log_2 n$深度的rank序列與重複find成本。列$\alpha(0),\alpha(2),\alpha(3),\alpha(7),\alpha(8),\alpha(2047),\alpha(2048)$，說明為何完整定理用$O(m(1+\alpha))$。

21-E09 代表最小值與有向圖

若應用要求查「集合最小鍵」，為何不可直接規定每次讓較小鍵根當parent後仍引用rank定理？提出保留rank根的O(1)附加更新規則。再用只有a→b的有向圖說明忽略方向的UNION不能求SCC；若邊可刪除，原DSU三操作為何不能直接維持正確連通性？

21-E10 iter 重設與沿路 level 的反例

同 rank 保留第一根。「分輪建立區間」指先合併相鄰單點，再合併相鄰同大小區塊，直到只剩區間左端為根。

1. 取 32 個單點。分輪建立 0..15，做 FIND(12)；建立 16..31，做 UNION(16,0)。列再次 FIND(12) 的路徑、各點 rank，以及壓縮前後的 level、iter、位能。說明 iter 下降為何不使位能增加。
2. 改取 16 個單點。分輪建立 0..7，做 FIND(2)；建立 8..15，做 UNION(8,0)。列再次 FIND(2) 前各點的 rank 與 level。這是否反駁「固定節點的 level 隨時間不下降」？
3. 兩例各有幾個好節點？分別比較實際個數 G 與下界 $g = \max(0, s - (\alpha + 2))$。沒有好節點是否代表此演算法錯誤？

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study